DAILY DATA DEEP DIVES · THE LATEST NEWS, FULLY EXPLAINED

Storage Wars

Postgres 19 reaches release candidate, and ClickHouse finds a 32 ms ghost in TCP

Issue #12 | Fri, 16 Oct 2026 | ⏱️ ~39 min read | Level 3/5

🗄️ Today's two stories

1. **PostgreSQL 19 RC 1 is out, and GA has moved to 19 November.** The release candidate shipped on 15 October with a list of last-minute fixes that is worth reading closely: a planner shortcut that could return wrong rows, a `REPACK` bug that turned real values into NULLs, and a `postgres_fdw` hole that could leak remote data through statistics. We explain what each fix tells you about the new features, the new schedule, and how to test this month.
2. **ClickHouse traced a mysterious 32 ms of extra latency to the Linux TCP stack, not the database.** After a connection sat idle for two seconds, a simple query took two network round trips instead of one. The cause was a chain of small socket writes, a congestion window that shrinks while you're idle, and a kernel setting most people never touch. We explain TCP slow start, Nagle's algorithm and the fix, and why the same trap can hit any database client on a long link.
3. **On the radar:** 7 more things that happened, listed at the end.

STORY 1 PostgreSQL 19 RC 1: what the last fixes reveal

POSTGRESQL 19 · RC 1 RELEASED 15 OCT 2026 · RC 2 AND GA PLANNED FOR 12 AND 19 NOV · ~16 MIN READ

A release candidate is the moment a database project says "we think this is it". PostgreSQL 19 reached that point yesterday, later than the project hinted in September, and with its general availability pushed to late November. The headline features haven't changed since we covered Beta 4 in Issue #1. What's new is the list of bugs fixed on the way to RC 1, and it's an unusually good guide to the new features. Each fix shows you where a feature's sharp edges were, and which parts of your workload deserve a test before you upgrade.

WHAT HAPPENED

On **15 October 2026**, the PostgreSQL Global Development Group released **PostgreSQL 19 RC 1**. The announcement says the RC "will be mostly identical to the initial release of PostgreSQL 19, though some more fixes may be applied prior to the general availability".[1]

The schedule changed with it. The next routine bug-fix release for all supported versions is on **12 November 2026**, and the release team says it "will include a large number of fixes". Rather than ship 19.0 without them, the team decided to fold them into 19 first. So a second release candidate, **RC 2**, will come out on 12 November, and the planned general availability date is **19 November 2026**, "unless an issue is discovered that requires a delay or an additional release candidate".[1]

That's later than the project expected a few weeks ago. The Beta 4 announcement in September said the RC "should occur in early October" and that GA "may also occur in October".[2] The PostgreSQL 19 open-items page on the project wiki now lists RC 1 on 15 October, RC 2 on 12 November and GA on 19 November.[3]

Cloud providers moved fast. On the same day, AWS put RC 1 into the **Amazon RDS Database Preview Environment**. Preview instances live for up to 60 days, are billed at normal RDS for PostgreSQL rates in US East (Ohio), and their snapshots can only be restored inside the preview environment.[4]

The RC 1 announcement lists the main fixes made since Beta 4:[1]

- several fixes for `REPACK (CONCURRENTLY)`;
- several fixes for `pg_plan_advice`;
- several fixes for "new query planner optimizations that rely on whether an expression can be `NULL`";
- several fixes for the automatic enablement of logical replication when `wal_level` is set to `replica`;
- `postgres_fdw` statistics import now runs as the owner of the foreign table, not the user running `ANALYZE`;
- several fixes for parallel autovacuum;
- `COPY (query) TO ... (FORMAT json)` now uses the query's column names, including aliases, as JSON keys.

The rest of this story walks through those items one by one: what the feature is, what went wrong, and what it means for you.

THE BACKGROUND YOU NEED

Release candidates, and why one release waits for another

PostgreSQL ships one **major version** *≈ a feature release, such as 19, that can change on-disk formats, so upgrading needs `pg_upgrade` or a dump and restore* a year and supports each for five years. Between major versions come **minor releases** *≈ bug- and security-fix releases, such as 18.6, that you install by swapping binaries and restarting*. The versioning page lists 18.6, 17.11, 16.15, 15.19 and 14.24 as the current minors, and **14's final release date is 12 November 2026**, the same day as RC 2.[5]

A new major version is developed in public: a **feature freeze** *≈ the date after which no new features are accepted for a release, only fixes*, then betas, then one or more **release candidates** *≈ builds the project believes are ready to ship, published so users can find any last serious bug*. For 19, the wiki lists the feature freeze on 8 April 2026, Beta 1 on 4 June, Beta 2 on 16 July, Beta 3 on 13 August and Beta 4 on 24 September.[3]

Why tie 19.0 to the November minor release? Many bug fixes land in every supported branch at once. If 19.0 shipped in October, a bug fixed in the next 18.x release on 12 November could still be present in 19.0, released weeks earlier. Holding GA for a week after that release means 19.0 starts life with every fix the older branches got.[1]

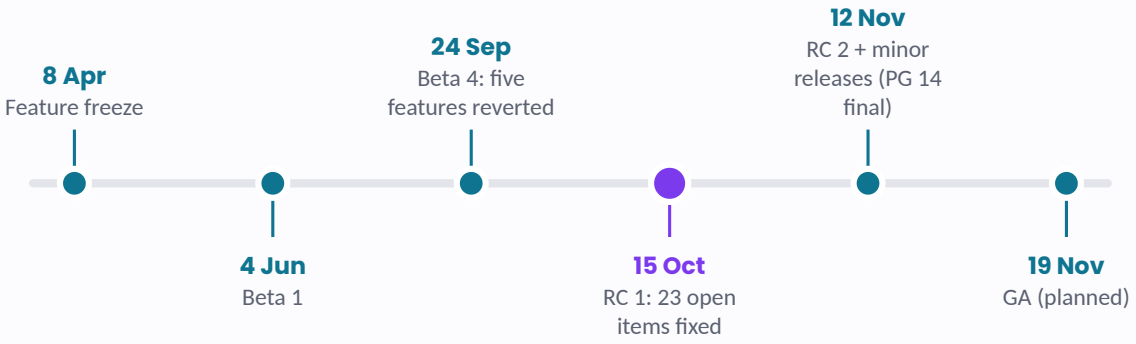
The open-items list

The project tracks release blockers on a public wiki page called **Open Items**. Each entry names the bug, the commit that introduced it, an owner, and the fixing commit. As of yesterday the page listed **23 items resolved**

before RC 1 and **3 still open**: a "megathread" of `pg_plan_advice` bugs, a failure of `DROP DATABASE ... WITH (FORCE)` when parallel autovacuum workers are running, and a case where the new fast path for foreign-key checks behaves differently from the old path when a foreign-key value needs a type cast.[3]

That page is what we'll use as a map. Most of the resolved items cluster around five new features.

PostgreSQL 19's road to release



Dates from the PostgreSQL 19 open-items wiki, the RC 1 announcement and the versioning policy, all viewed 16 Oct 2026.

HOW IT WORKS: FIVE FEATURES AND THE BUGS THAT TESTED THEM

1. The planner learns to trust NOT NULL, and briefly trusted it too much

This is the most interesting set of fixes, because the feature is invisible. You won't change any SQL to get it, and a bug in it shows up as wrong results, not an error.

Start with a classic SQL trap. `NOT IN` with a subquery behaves strangely when NULLs are involved. The PostgreSQL docs put it precisely: if "there are no equal right-hand values and at least one right-hand row yields null, the result of the `NOT IN` construct will be null, not true."[7] Since a `WHERE` clause only keeps rows where the condition is true, one NULL in the subquery makes the whole query return nothing:

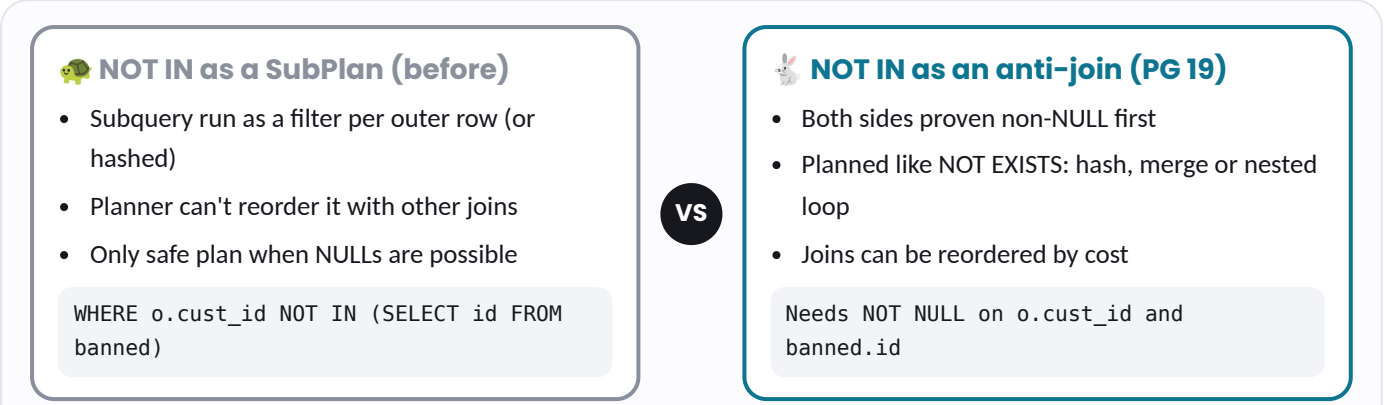
```
-- No rows: is 1 unequal to NULL? "Unknown", so the predicate is never true.
SELECT 1 WHERE 1 NOT IN (SELECT NULL::int);
```

Because of that rule, the **query planner** *≈ the part of Postgres that turns your SQL into an execution plan, choosing join order, join method and indexes from cost estimates* has historically treated `NOT IN (SELECT ...)` as a black box. The commit that changed this explains why. Standard SQL says that if `x = y` returns NULL, `NOT IN` evaluates to NULL and the row is dropped. "In contrast, an **anti-join** *≈ a join that returns rows from the left side that have no match on the right side; the plan shape behind NOT EXISTS* preserves the row if no match is found."[8] So the planner ran `NOT IN` as an opaque **SubPlan** *≈ a subquery executed as a filter attached to the outer query, rather than planned as a join* filter. It couldn't reorder it or pick a hash join for it.

PostgreSQL 19 lifts that restriction when it's provably safe. The commit, by Richard Guo in March, says that "if we can prove that neither side of the comparison can yield NULL values, and further that the operator itself cannot return NULL for non-null inputs, the behavior of NOT IN and anti-join becomes identical." It then lets

the planner treat the subquery "as a first-class relation", which "unlocks global join ordering optimization and permits the selection of the most efficient join algorithm based on cost".[8]

How does it prove "can't be NULL"? It uses several sources: `NOT NULL` constraints on the table, whether a column comes from the nullable side of an outer join, and `WHERE` clauses that rule out NULLs (for example `col > 0` can't be true for a NULL). For the operator, it requires that it belong to a **B-tree or hash operator family** ≈ *the sets of comparison operators an index type knows how to use; membership implies the operator gives true or false, never NULL, for non-NULL inputs*.[8]



Based on commit 383eb21 (pgsql-committers). The example query is illustrative.

The release notes list a family of related rewrites, all by Richard Guo: `IS [NOT] DISTINCT FROM NULL` becomes `IS [NOT] NULL`; `IS [NOT] DISTINCT FROM` becomes plain `=` / `<>` when inputs are proven non-nullable; `COALESCE()` and `ROW(...) IS [NOT] NULL` skip arguments that can't matter; and `IS TRUE/FALSE/UNKNOWN` simplifies to a plain boolean when the input can't be NULL.[6]

Now the bugs. Proving "never NULL" is harder than it sounds, and the open-items page lists three fixes in this area before RC 1: an `ON CONFLICT` that missed a unique expression index on a `NOT NULL` column, a `COALESCE` on a `NOT NULL` column that lost its **collation** ≈ *the rules a database uses to compare and sort text, such as case sensitivity and language-specific ordering* and type modifier, and a `NOT IN` turned into an anti-join over a virtual generated column that "can read as NULL".[3]

The last one is a lovely edge case. A **virtual generated column** ≈ *a column whose value is computed from other columns each time it's read, rather than stored on disk* can be declared `NOT NULL`. But the fix commit, pushed on 10 October, explains that in a **partitioned table** ≈ *one logical table stored as several physical child tables, called partitions, each holding a range or list of keys* "a partition can have a generation expression different from its parent's", and enforces `NOT NULL` against its own expression, while a query on the parent computes the column with the parent's expression. "So the column can be NULL when read through the parent despite the constraint, and the anti-join would then produce wrong results, as it returns rows that NOT IN should have rejected." The fix makes the planner ignore that particular `NOT NULL` constraint.[9]

What should you take from this? First, the optimization is good news for anyone with legacy `NOT IN` queries. Declaring columns `NOT NULL` where they really are never NULL now buys you better plans, not just data quality. Second, wrong-result bugs are the scariest kind, and this whole area is new. If you have `NOT IN`, `IS DISTINCT FROM` or `COALESCE` in hot queries, compare their results on 18 and on the RC with real data.

2. REPACK (CONCURRENTLY): a NULL where a value should be

We explained `REPACK` in Issue #1. In short, it replaces `VACUUM FULL` and `CLUSTER`, and `REPACK (CONCURRENTLY)` rewrites a table without blocking reads and writes for most of the run.[6] It does that by copying the table, recording the changes your application makes in the meantime with **logical decoding** ≈ *turning the write-ahead log back into row-level changes, the machinery behind logical replication*, replaying those changes onto the new copy, and only taking a strong lock for the final swap.

The RC 1 announcement lists "several fixes" for it, and the open-items page has several `REPACK` entries.[1,3] One shows how subtle the replay step is. It involves a Postgres trick you may use without knowing it: when you run `ALTER TABLE ... ADD COLUMN` with a constant default, Postgres doesn't rewrite the table. Old rows simply have fewer columns on disk, and the default is stored once as a "missing" value that's filled in when the row is read.

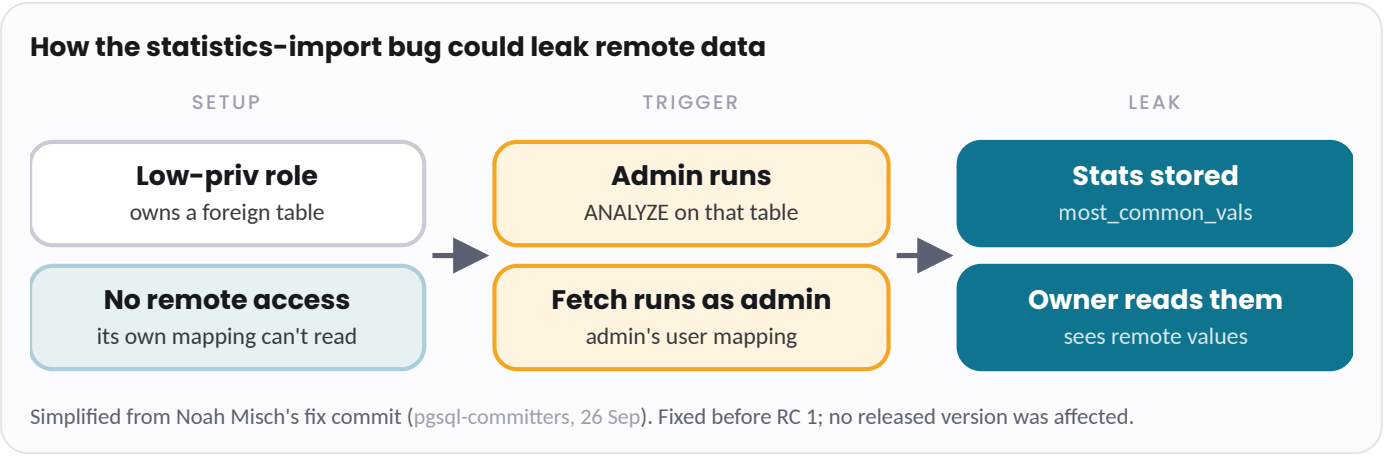
The fix commit, dated 25 September, describes the bug. When `REPACK` replayed concurrent changes, it decoded rows using the description of the new, temporary table, and that description "has no 'missing' values". So a row written before such an `ADD COLUMN` came out with NULL in the new column, "even in a column declared NOT NULL". If that column was part of the **replica identity** ≈ *the columns Postgres uses to identify which row an UPDATE or DELETE refers to when replaying changes; usually the primary key*, the lookup key was NULL and the target row couldn't be found, and with some data types the comparison could even crash. The commit says such rows "will be rare" and names one way to produce one: a `BEFORE ROW UPDATE` trigger that returns `OLD`.[10]

The practical lesson is that `REPACK (CONCURRENTLY)` is new machinery doing delicate work on your live data. Use it on 19.0 the way you'd use any new tool on a busy table: try it on a copy first, and check row counts and a few checksums afterwards.

3. postgres_fdw statistics import: a privilege leak, caught in time

`postgres_fdw` is the **foreign data wrapper** ≈ *an extension that makes a table on another server look like a local table you can query* for talking to other Postgres servers. To plan queries on a **foreign table** ≈ *a local table definition whose rows actually live on a remote server*, Postgres needs **planner statistics** ≈ *summaries of a table's data, such as most common values and histograms, that the planner uses to estimate row counts*. Until now, `ANALYZE` on a foreign table pulled sample rows across the network and computed statistics locally. PostgreSQL 19 lets it fetch the remote server's existing statistics instead, which is much cheaper for big tables.[6]

The fix, committed on 26 September by Noah Misch, closed a real security hole. When importing statistics, `ANALYZE` ran "as the user executing ANALYZE, not as the foreign table's owner". That meant a role that owned a foreign table but had no access to the remote data "could obtain that data by having a privileged user run ANALYZE on the table". Statistics such as `most_common_vals` contain actual values, so they would "expose sampled values from the remote data". The fix switches to the table owner's identity, the same one the row-sampling path already used.[11]



Simplified from Noah Misch's fix commit (pgsql-committers, 26 Sep). Fixed before RC 1; no released version was affected.

One oddity to watch: the release notes (as of 9 October) say the feature "is enabled for `postgres_fdw` by using the option `import_stats`".[6] The `postgres_fdw` documentation page instead describes an option called `restore_stats`, settable per server or per table, default `false`, which falls back to row sampling if fetching fails and makes it "the user's responsibility" to keep remote statistics fresh.[12] Check which name the final 19.0 docs use before you put it in a migration script.

4. Logical replication without a restart, and two corner cases

Postgres's **write-ahead log (WAL)** *≈ the append-only log where Postgres records every change before touching data files; replicas and CDC tools read it* can be written at different levels of detail. The `wal_level` setting's default, `replica`, is enough for backups and physical replicas. **Logical replication** *≈ copying row-level changes (rather than raw WAL bytes) to another server or a CDC tool* needs `logical`, which adds extra information. And `wal_level` "can only be set at server start".[13] For years, that meant a restart before you could attach Debezium or set up a logical-replication upgrade.

PostgreSQL 19 changes the rule. With `wal_level = replica`, "the system automatically increases the effective WAL level to `logical` when creating the first logical **replication slot** *≈ a bookmark in the WAL that stops Postgres from deleting log segments a consumer hasn't read yet*, and decreases it back to `replica` when dropping or invalidating the last" one. A new read-only setting, `effective_wal_level`, reports what's actually in effect, and on a standby it matches "the most upstream server in the replication chain".[13,14]

Switching levels on a running server means every session has to notice. Two fixes before RC 1 were exactly that: a "session in aborted transaction misses effective_wal_level change", and an issue in **slot synchronization** *≈ copying logical replication slots from a primary to its standby so CDC can continue after a failover* "when logical decoding is disabled and re-enabled".[3]

Turning on CDC: PostgreSQL 18 vs 19

PostgreSQL 18

```
-- postgresql.conf
wal_level = logical
-- then: restart the server
SELECT pg_create_logical_replication_slot(
  'cdc', 'pgoutput');
```

PostgreSQL 19 (wal_level = replica)

```
SHOW effective_wal_level;  -- replica
SELECT pg_create_logical_replication_slot(
  'cdc', 'pgoutput');
SHOW effective_wal_level;  -- logical
-- drop the last slot: back to replica
```

Behaviour from the PG 19 wal_level and effective_wal_level docs. Slot name and plugin are illustrative.

What it means for you: a forgotten logical slot now has two costs. It already held back WAL deletion. In 19 it also keeps your whole server writing the more verbose `logical` level, which the docs warn "will increase the WAL volume".[13] Add `effective_wal_level` to your monitoring next to slot lag.

5. Parallel autovacuum, and the faster COPY to JSON

Autovacuum ≈ *the background process that runs VACUUM and ANALYZE on tables automatically when enough rows have changed* in 19 can use parallel workers for a table's indexes. Manual `VACUUM` has had a `PARALLEL` option for a while; its rules are that an index qualifies only if it's larger than `min_parallel_index_scan_size`, and "only one worker can be used per index", so workers launch only when a table has at least two indexes.[15] The new server setting `autovacuum_max_parallel_workers` brings this to autovacuum. It applies only to the index vacuuming and index cleanup phases, is "further limited by max_parallel_workers", and **defaults to 0, meaning off**.[16] A per-table storage parameter, `autovacuum_parallel_workers`, overrides it (`-1` means use the server setting, `0` disables it for that table).[17]

Two parallel-autovacuum bugs show the kind of coordination involved. One fixed before RC 1: the leader process didn't pass on changes to cost-limit settings to its workers while waiting for them. One still open: `DROP DATABASE ... WITH (FORCE)` fails when parallel autovacuum workers are attached to the database.[3]

Finally, a small, pleasant feature with a small bug. 19 adds `COPY ... TO ... (FORMAT json)`, which writes one JSON object per row, plus a `FORCE_ARRAY` option that wraps everything in one array. It's `COPY TO` only, and it can't be combined with `HEADER`, `DELIMITER` or `NULL`.[18] The depesz blog notes that the old workaround, `COPY (SELECT to_json(t) FROM t) TO ...`, could emit invalid JSON with doubled backslashes; the native format handles those rows correctly.[19] The bug fixed on 8 October: when the query didn't need to reshape rows, keys came from the table, "losing column aliases and mislabelling rows from UNION ALL branches with different column orders".[20]

```
-- One JSON object per line (NDJSON); keys now follow the aliases
COPY (SELECT id, email AS contact FROM users) TO STDOUT (FORMAT json);

-- One big array instead
COPY users TO STDOUT (FORMAT json, FORCE_ARRAY);
```

One more detail from the docs: a SQL NULL and a JSON column holding the JSON literal `null` both come out as `null`, so the two are "indistinguishable in the `COPY` output".[18]

WHAT IT MEANS FOR YOU

If you run PostgreSQL 14, this is your deadline. Its last release is on 12 November, one week before 19.0.[5] Whether you jump to 18 now or wait for 19 depends on how quickly your provider offers 19. RDS already has the RC in its preview environment.[4]

If you plan to adopt 19 early, test the RC this month, and aim your testing at the areas the open-items page says were fragile:

1. **Query results, not just speed.** Run your important queries on 18 and on the RC and diff the output, especially anything with `NOT IN`, `IS DISTINCT FROM`, `COALESCE`, outer joins, partitioned tables and virtual generated columns.
2. `REPACK (CONCURRENTLY)` **on a copy** of a real busy table, including tables that had columns added over the years.
3. **CDC tooling** (Debezium, logical replication) with `wal_level = replica`, watching `effective_wal_level`.
4. `postgres_fdw` **users:** check the statistics option's final name, and who owns your foreign tables.
5. **Foreign keys whose referencing column needs a type cast** to compare with the referenced key, because the new fast path for foreign-key checks still has an open item about casts.[3] The thread behind it explains the risk plainly: without a fix, "a foreign key using such a cast is checked differently depending on whether the fast path or SPI handles it."[21]

If you're on 18 and happy, nothing changes yet. Managed services usually add a new major version some time after GA, and many teams wait for the first bug-fix release anyway.

> 💥 **Watch out for**
>
> - **RC is not GA.** RC 2 is already scheduled with "a large number of fixes" from the November minor releases. Don't run RC 1 in production.[1]
> - **Getting onto the RC is a major upgrade.** The announcement says upgrading to RC 1 from earlier versions needs `pg_upgrade` or dump/restore, so test on a copy, not in place.[1]
> - **Three items are still open,** including the `pg_plan_advice` megathread. If you were counting on `pg_plan_advice` from day one, follow that thread.[3]
> - **RDS preview instances are deleted after 60 days,** and their snapshots can't leave the preview environment.[4]
> - `postgres_fdw` **option name:** release notes say `import_stats`, docs say `restore_stats`.[6,12]

WHAT HAPPENS NEXT

RC 2 and the November minor releases arrive on 12 November, and GA is planned for 19 November.[1] Watch the open-items page: if one of the three remaining items turns into a release blocker, the schedule says another RC would follow.[3] We'll cover 19.0 when it ships, with a focus on what changed between RC 1 and GA.

📖 Words from this story

Major version: a feature release (19) that can change on-disk formats; upgrading needs `pg_upgrade` or dump/restore.

Minor releases: bug- and security-fix releases (18.6) installed by swapping binaries.

Feature freeze: the date after which a release accepts only fixes.

Release candidates: builds believed ready to ship, published to catch any last serious bug.

Query planner: turns SQL into an execution plan, choosing join order, methods and indexes.

Anti-join: returns left-side rows with no match on the right; the plan behind `NOT EXISTS`.

SubPlan: a subquery run as a filter attached to the outer query, not planned as a join.

B-tree or hash operator family: operator sets an index type can use; implies no NULL result for non-NULL inputs.

Collation: the rules for comparing and sorting text.

Virtual generated column: a column computed from other columns each time it's read.

Partitioned table: one logical table stored as several child tables by key range or list.

Logical decoding: turning WAL back into row-level changes.

Replica identity: the columns used to find the target row when replaying an UPDATE or DELETE.

Foreign data wrapper: an extension that makes a remote table look local.

Foreign table: a local table definition whose rows live on a remote server.

Planner statistics: summaries of a table's data used to estimate row counts.

Write-ahead log (WAL): the append-only change log that replicas and CDC tools read.

Logical replication: copying row-level changes to another server or CDC tool.

Replication slot: a WAL bookmark that stops Postgres deleting log a consumer still needs.

Slot synchronization: copying logical slots to a standby so CDC survives failover.

Autovacuum: background VACUUM/ANALYZE triggered by table activity.

STORY 2 The 32 ms ghost: when TCP, not the database, is slow

CLICKHOUSE · ENGINEERING BLOG, 13 OCT 2026 · NATIVE PROTOCOL FIX IN PR #111034 · ~14 MIN READ

A customer reported something that sounds impossible. Their ClickHouse server sat about 32 ms away, so a tiny query should take about 32 ms. Usually it did. But if the connection had been idle for two seconds or more, the next query took about 64 ms, almost exactly twice as long. The query was trivial, the server wasn't busy, and the extra time always matched one network round trip. ClickHouse engineer Kavi Kanagaraj tracked it down, and the culprit turned out to be a conversation between ClickHouse's own socket writes and a piece of TCP behaviour that dates back to the year 2000. It's one of the best "it's not the database" stories in a while, and the same trap can catch any client that talks to a database across a long link.

WHAT HAPPENED

On **13 October 2026**, ClickHouse published "Chasing that 32 ms tail latency in ClickHouse". In the customer's setup, "the client and server are ~32 ms apart". When "a connection sat idle for 2 seconds or longer, the very next query took ~64 ms end to end", which is two **round-trip times (RTT)** ≈ *the time for a message to travel from one machine to another and for the reply to come back* instead of one.[22]

The team reproduced it in a lab, found the cause in the interaction between ClickHouse's **Native protocol** ≈ *ClickHouse's own binary client-server protocol over TCP, used by clickhouse-client and most drivers; the alternative is HTTP* and the Linux TCP stack, and changed the server to group its end-of-query messages into a single socket write. The change is in ClickHouse pull request #111034. In the lab, the extra delay fell from about 32 ms to about 1 ms.[22] The post doesn't say which ClickHouse release will contain the fix; its reproduction used a build older than 26.9.[22]

THE BACKGROUND YOU NEED

How a ClickHouse query comes back

When a client sends a query over the Native protocol, the server replies with a stream of typed packets. The protocol documentation lists them: `Data` (a block of result rows), `Progress` ("query execution progress"), `ProfileInfo`, `Log` (server log lines), `ProfileEvents` ("per-query performance counters"), `EndOfStream` (the query is complete) and a few others.[23]

The important detail is that the client can't finish until the last one arrives. As the blog puts it, "clients must wait for an `EndOfStream` packet to complete a query." So the end-to-end time isn't "when did the first row arrive" but "when did `EndOfStream` arrive". The blog measures both: `first_row`, and `tail`, the gap between the first row and the end of the stream. A healthy tail is close to 0 ms; a stalled one equals a full RTT.[22]

TCP's speed limit: the congestion window

TCP doesn't let a sender blast unlimited data into the network. It keeps a **congestion window (cwnd)** ≈ *the amount of data a TCP sender may have in flight, sent but not yet acknowledged; it grows as the connection proves the network can take more* . Once that much data is in flight, the sender must stop and wait for an **ACK** ≈ *a TCP acknowledgement: the receiver telling the sender which data has arrived, which frees room in the window* , and an ACK takes one round trip to come back.

A new connection starts with a small window and grows it quickly, a phase called **slow start** ≈ *TCP's start-up phase, in which the congestion window begins small and roughly doubles every round trip until the network shows signs of congestion* . The starting size is the **initial window (initcwnd)** ≈ *the congestion window a TCP connection starts with, measured in maximum-size segments* . RFC 6928, published in April 2013, raised the standard upper bound to 10 segments, "min (10*MSS, max (2*MSS, 14600))", where an **MSS** ≈ *maximum segment size: the largest chunk of data that fits in one TCP segment on that path, typically about 1.4 KB on Ethernet* is one full-size **TCP segment** ≈ *one TCP packet's worth of data, with its own header* .[24] The ClickHouse post notes that 10 is the common default, but in the customer's environment `initcwnd` was 8.[22]

Why an idle connection forgets its speed

Here's the part almost nobody knows about. A window learned a minute ago may be wrong now, because other traffic may have filled the network. RFC 2861, from June 2000, proposed that a sender which stops sending should not trust its old window. It describes halving the window for each period of idleness, while remembering the old value so it can climb back quickly.[25] (That RFC has since been marked historic and obsoleted by RFC 7661, but the idea lives on in Linux.)

Linux has a setting for this, `net.ipv4.tcp_slow_start_after_idle` . The kernel documentation for it says: "If set, provide RFC2861 behavior and time out the congestion window after an idle period. An idle period is defined at the current RTO." The default is **1, on**.[26] The **RTO** ≈ *retransmission timeout: how long TCP waits for an ACK before resending, derived from measured RTTs* is TCP's own estimate of how long a reply should take, so it scales with the link. The ClickHouse post summarizes the result: after the connection sat idle for about two seconds, Linux shrank `cwnd` back to `initcwnd` .[22]

So every query after a pause starts with a window of 8 segments, as if the connection were brand new.

Nagle's algorithm and TCP_NODELAY

One more piece. When a program writes a few bytes to a TCP socket, should the kernel send them right away or wait to collect more? By default, Linux uses **Nagle's algorithm** ≈ *a TCP rule that holds back small writes while earlier data is unacknowledged, so many tiny writes are combined into fewer, fuller packets* : "data is buffered until there is a sufficient amount to send out, thereby avoiding the frequent sending of small packets".[27]

Latency-sensitive software, databases included, usually turns that off with the `TCP_NODELAY` socket option, which means "segments are always sent as soon as possible, even if there is only a small amount of data".[27] That's the right call for interactive traffic, because Nagle's waiting can add delays of its own. But it has a side

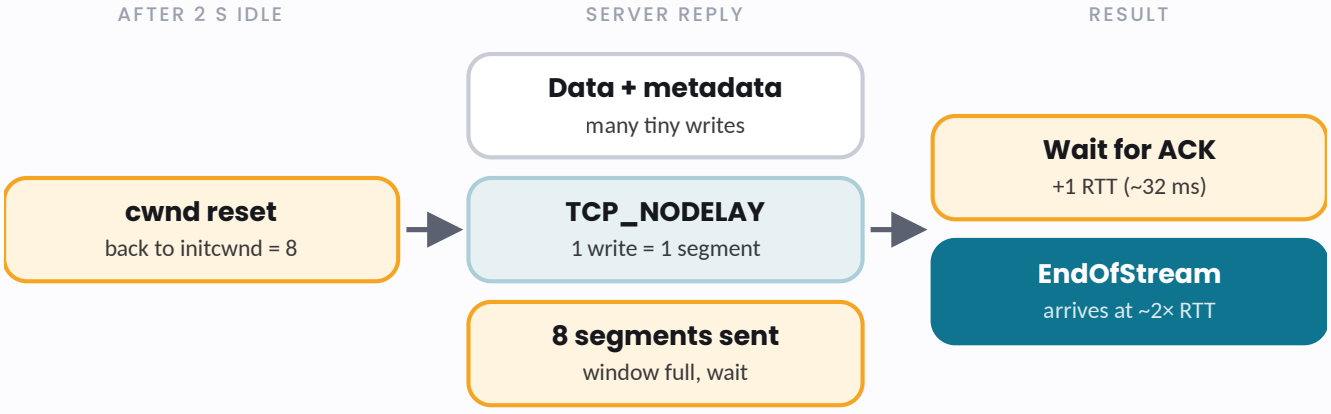
effect: every separate write becomes its own TCP segment. ClickHouse uses `TCP_NODELAY`, so "each one becomes a separate TCP segment".[22]

HOW IT WORKS: THE ANATOMY OF A 32 MS STALL

Put the pieces together and the mystery solves itself.

1. The connection is idle for two seconds. Linux times out the congestion window, and `cwnd` drops back to 8 segments.[22,26]
2. The client sends `SELECT max(id) ...`. The server runs it in well under a millisecond.
3. The server writes its reply: a `Data` packet with the result, then a series of small metadata packets (`ProfileEvents`, `Progress`, `Logs`), then `EndOfStream`. Each was a separate socket write, and with `TCP_NODELAY` each left as a separate segment. The packet capture showed many tiny segments, between 1 and 36 bytes.[22]
4. The window counts segments, not usefulness. The blog's diagnosis: "The server sent 8 TCP segments, ran out of `cwnd` space and paused until the client ACKed the first batch".[22]
5. The client gets the first row after one RTT, but the rest of the reply, including `EndOfStream`, is still sitting in the server's send buffer. It leaves only when the ACK arrives, one more round trip later. Total: about 2 × RTT.

Why the reply needed two round trips after an idle pause



Sequence from ClickHouse's post, 13 Oct 2026. Customer environment: ~32 ms RTT, initcwnd 8.

Notice how specific the conditions are. With a 1 ms RTT, the extra round trip would cost 1 ms and nobody would notice. With a busy connection, the window would stay large and the stall wouldn't happen. With Nagle's algorithm on, the small writes would have been merged. It took a long link, idle gaps and many small writes together.

Reproducing it on a laptop

The post's reproduction is worth copying if you ever need to test latency behaviour. It creates a **network namespace** *≈ a Linux feature that gives a set of processes its own isolated network stack, useful for simulating a remote machine on one host* for the server, connects it with a virtual Ethernet pair, adds 32 ms of delay with `tc netem`, and sets the initial window on the route:[22]

```
sudo ip netns exec chsrv tc qdisc add dev veth-ch root netem delay 32ms
sudo ip netns exec chsrv ip route change 10.99.0.0/24 dev veth-ch \
    proto kernel scope link src 10.99.0.2 initcwnd 8
```

A small Go program then ran `SELECT max(id)` repeatedly on one connection with two-second gaps, recording `first_row`, `eos` and `tail`. Against ClickHouse 26.8, every run looked the same: `first_row` about 33 ms, `eos` about 65 ms, and a tail of 31.5 to 32.1 ms.[22] A `tshark` capture counted about 50 data-carrying packets from server to client in the run.[22]

Five ways out

The team listed five possible fixes:[22]

1. turn off `ProfileEvents` on the server (`send_profile_events`);
2. set `tcp_slow_start_after_idle=0` on the server, so the window isn't timed out;
3. raise `initcwnd` on the server;
4. batch Native protocol packets in ClickHouse itself;
5. use `TCP_CORK` instead of `TCP_NODELAY`.

The customer had limited control over the first three, so the team picked the fourth.[22] The fifth is interesting. **TCP_CORK** *≈ a Linux socket option that holds back partial segments until the program "uncorks", so several writes leave as full packets* would make the kernel collect the small writes. The team rejected it because "the kernel TCP stack still has full control over filling those TCP segments" and "it is specific to the Linux kernel".[22] The man page adds that `TCP_CORK` "should not be used in code intended to be portable", and that there's "a 200 millisecond ceiling" on how long output stays corked.[27]

🖥️ Fix it in the OS

- tcp_slow_start_after_idle = 0
- Raise initcwnd on the route
- Helps every protocol on the host
- Needs control of the server's kernel

```
sysctl
net.ipv4.tcp_slow_start_after_idle=0
```

VS

📦 Fix it in the application

- Group end-of-query packets into one write
- Fewer, fuller segments fit in a small window
- Works on any OS, any kernel settings
- Time-sensitive packets still go out at once

```
ClickHouse PR #111034
```

Options from ClickHouse's post. The sysctl name is from the kernel documentation.

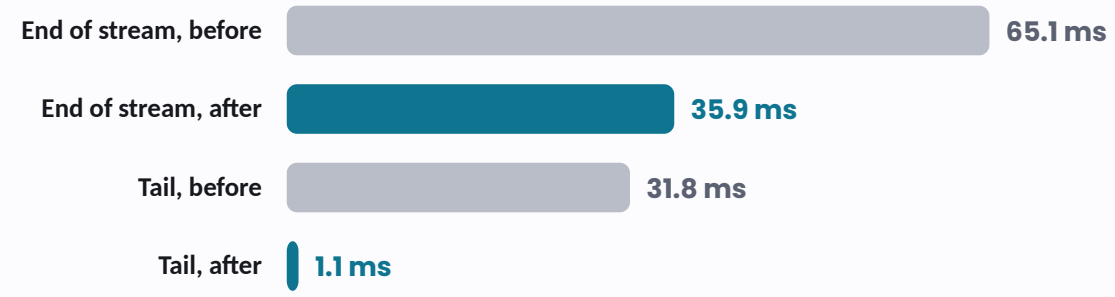
The fix: one write instead of many

ClickHouse now groups the metadata packets at the end of a query "at the application level into a single socket write". Fewer writes mean fewer segments, so the whole tail of the reply fits inside even a freshly reset window and arrives in the minimum number of round trips.[22]

There's a deliberate exception. "Some packets like `Progress` are time-sensitive and have to be delivered to the client immediately". If progress updates were held back, a client showing a progress bar would see stale numbers.[22] (The post is slightly inconsistent here: its summary lists `Progress` among the batched packets, while the body says it can't be grouped. The idea is clear either way: batch what can wait, send what can't.)

The results, from the same lab setup:[22]

Lab results, SELECT max(id) after 2 s idle, ~32 ms RTT (ms, mean of 5 runs)



Means computed by Storage Wars from the five before/after runs in ClickHouse's post. Vendor lab measurement in a network namespace, not a production benchmark.

The packet count for the run halved, from about 50 to about 25, and the tail dropped to between 0.8 and 1.3 ms.[22] The first row now arrives about 1–2 ms later than before (34.5–35.2 ms versus 33.2–33.7 ms in the post's tables), which the post doesn't comment on, while the end of the stream arrives about 29 ms sooner.[22]

WHAT IT MEANS FOR YOU

If you use ClickHouse over a long link, for example a client in one region and a server in another, or an app talking to ClickHouse Cloud across the internet, and your traffic comes in bursts with idle gaps, this was probably costing you one RTT on some queries. Watch the release notes for PR #111034. Until then, the post's workarounds apply, and the cheapest one you control may be `send_profile_events`.[22]

If you run your own servers, check `net.ipv4.tcp_slow_start_after_idle`. Turning it off is a known tweak for long-lived connections; the kernel commit that added it begins, "A lot of people have asked for a way to disable tcp_cwnd_restart()".[26] But it's a host-wide setting, and it exists for a reason: it stops a connection that has been quiet from bursting a large window into a network whose conditions may have changed.[25] Treat it as a decision, not a default to flip everywhere.

If you write database drivers or protocols, the lesson is general. `TCP_NODELAY` plus many small writes is a quiet multiplier: each write spends one slot of a window you don't control. Build each logical message, or better each group of messages that will be sent together, in a buffer, and write it once. The ClickHouse post closes with the principle: "High tail latency isn't always caused by slow database engines or long-running queries."[22]

And if you're debugging latency anywhere, keep this signature in mind. A delay that's an exact multiple of your RTT, appears only after idle periods, and doesn't change with server load is almost never the query. It's the network stack waiting for an ACK.

💥 Watch out for

- **No release version yet.** The post links the PR but doesn't say which ClickHouse version ships it.[22]
- **Lab numbers.** The before/after figures come from ClickHouse's own reproduction in a network namespace, not a customer production measurement.[22]
- **The HTTP interface isn't covered.** The post only discusses the Native protocol, though it notes HTTP is the other main way to talk to ClickHouse.[22]
- **Turning off slow-start-after-idle is host-wide.** It affects every TCP connection on the machine, not just your database.[26]
- `TCP_CORK` **isn't a portable fix,** and it has a 200 ms ceiling on Linux.[27]

📖 Words from this story

Round-trip time (RTT): time for a message to go and a reply to come back.

Native protocol: ClickHouse's binary client-server protocol over TCP.

Congestion window (cwnd): how much unacknowledged data a TCP sender may have in flight.

ACK: a TCP acknowledgement that frees room in the window.

Slow start: TCP's start-up phase where the window grows quickly from a small value.

Initial window (initcwnd): the window a connection starts with, in segments.

MSS: maximum segment size, the largest data chunk per TCP segment (~1.4 KB on Ethernet).

TCP segment: one TCP packet's worth of data.

RTO: retransmission timeout; how long TCP waits for an ACK before resending.

Nagle's algorithm: holds small writes back so they're combined into fuller packets.

Network namespace: an isolated Linux network stack, handy for simulating remote hosts.

TCP_CORK: a Linux option that holds partial segments until uncorked.

On the radar

Also happened, in one line each. Some will get a full deep dive in a future issue.

- **Snowflake Postgres data mirroring is GA:** tables in a Snowflake Postgres instance can now be continuously mirrored into a Snowflake database, including schema changes, on AWS and Azure (15 Oct).[28]
- **Databricks (13–15 Oct):** external engines can now query Unity Catalog views through the Unity REST API (GA), and the ADBC driver became the GA default for the Power BI connector.[29]
- **Kafka 4.4.0 is still stuck in release candidates:** RC1 (22 Sep) and RC3 (30 Sep) both had release problems such as missing artifacts and invalid signatures, and the downloads page still tops out at 4.3.1.[30,31]
- **Kafka KIP-1376 proposes configurable TLS named groups,** which would let clients and brokers use post-quantum key exchange on JVMs that support it.[30]
- **Debezium 3.7 (29 Sep)** added new connectors for TiDB, Milvus and SQLite.[30]
- **MongoDB driver advisory:** Canada's Cyber Centre flagged fixes in the MongoDB Go driver (2.9.0/2.9.2) and libmongocrypt (1.20.5) for BSON length checks and duplicate key-encryption-key fields (9 Oct).[32]
- **Planet Valkey launched (14 Oct):** a new aggregator collecting Valkey news and blog posts from the project, vendors and cloud providers.[33]

1. **PostgreSQL 19 RC 1 Released!**, PostgreSQL Global Development Group, 15 Oct 2026.
2. **PostgreSQL 19 Beta 4 announcement text**, PostgreSQL Global Development Group, Sep 2026.
3. **PostgreSQL 19 Open Items**, PostgreSQL wiki, viewed 16 Oct 2026.
4. **PostgreSQL 19 RC1 is now available in Amazon RDS Database Preview Environment**, AWS What's New, 15 Oct 2026.
5. **Versioning Policy**, PostgreSQL, viewed 16 Oct 2026.
6. **PostgreSQL 19 release notes**, "as of 2026-10-09", viewed 16 Oct 2026.
7. **Subquery Expressions: NOT IN**, PostgreSQL 19 documentation, section 9.25.3.
8. **pgsql: Convert NOT IN sublinks to anti-joins when safe**, Richard Guo, pgsql-committers, 12 Mar 2026.
9. **pgsql: Don't trust NOT NULL on virtual generated columns of partitioned tables**, pgsql-committers, REL_19_STABLE, 10 Oct 2026.
10. **pgsql: Fix REPACK (CONCURRENTLY) for columns added without a table rewrite**, pgsql-committers, REL_19_STABLE, 25 Sep 2026.
11. **pgsql: Fix privilege handling in postgres_fdw statistics import**, Noah Misch, pgsql-committers, REL_19_STABLE, 26 Sep 2026.
12. **postgres_fdw**, PostgreSQL 19 documentation, section F.40, viewed 16 Oct 2026.
13. **Write Ahead Log settings: wal_level**, PostgreSQL 19 documentation, section 19.5.1.
14. **Preset Options: effective_wal_level**, PostgreSQL 19 documentation.
15. **VACUUM**, PostgreSQL 16 documentation, PARALLEL option.
16. **Vacuuming settings: autovacuum_max_parallel_workers**, PostgreSQL 19 documentation.
17. **CREATE TABLE: storage parameters**, PostgreSQL 19 documentation.
18. **COPY**, PostgreSQL 19 documentation.
19. **Waiting for PostgreSQL 19 – json format for COPY TO**, depesz, 29 Mar 2026.
20. **pgsql: COPY TO FORMAT JSON: use the query's column names as keys**, Andrew Dunstan, pgsql-committers, 8 Oct 2026.
21. **Re: PG19: two RI fast-path issues found while testing the batching revert**, Amit Langote, pgsql-hackers, 7 Oct 2026.
22. **Chasing that 32 ms tail latency in ClickHouse**, Kavi Kanagaraj, ClickHouse blog, 13 Oct 2026.
23. **Native protocol: server packets**, ClickHouse documentation, viewed 16 Oct 2026.
24. **RFC 6928: Increasing TCP's Initial Window**, J. Chu et al., IETF, April 2013.
25. **RFC 2861: TCP Congestion Window Validation**, M. Handley et al., IETF, June 2000 (obsoleted by RFC 7661).
26. **Linux commit adding tcp_slow_start_after_idle**, including its ip-sysctl documentation entry (Default: 1).
27. **tcp(7) man page**, Linux man-pages, TCP_NODELAY and TCP_CORK.
28. **Data mirroring for Snowflake Postgres (General availability)**, Snowflake documentation, 15 Oct 2026.
29. **Databricks platform release notes, October 2026**, entries dated 13 and 15 Oct 2026.
30. **Kafka Monthly Digest: September 2026**, Red Hat Developer, 5 Oct 2026.
31. **Apache Kafka downloads**, viewed 16 Oct 2026.
32. **MongoDB security advisory (AV26-1024)**, relaying the Canadian Centre for Cyber Security, 9 Oct 2026.
33. **Announcing Planet Valkey**, Jean-François Gagné, Valkey blog, 14 Oct 2026.